



Architecting MLOps Solutions

LECTURE

Opinionated MLOps Principles

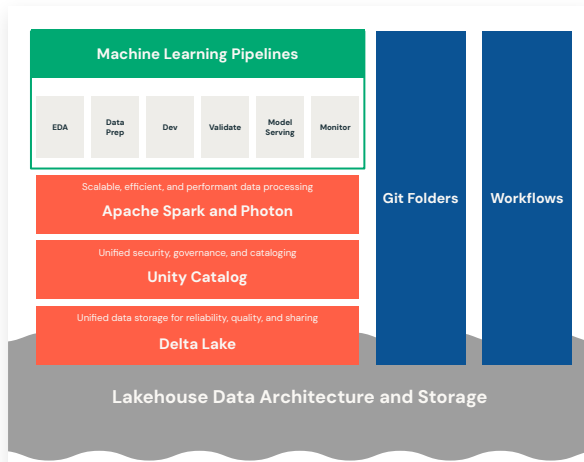


© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

The lecture **Opinionated MLOps Principles** covers guiding principles, environment semantics, separation, and deployment patterns for streamlined ML operations.

Guiding Principle

A data-centric approach to machine learning



- ML projects are made up of data pipelines
- Operationalizing ML solutions requires the connection of a variety of data pipelines
- Data pipelines require storage, governance, orchestration, etc.
- Aligned to the **Data Intelligence Platform vision**

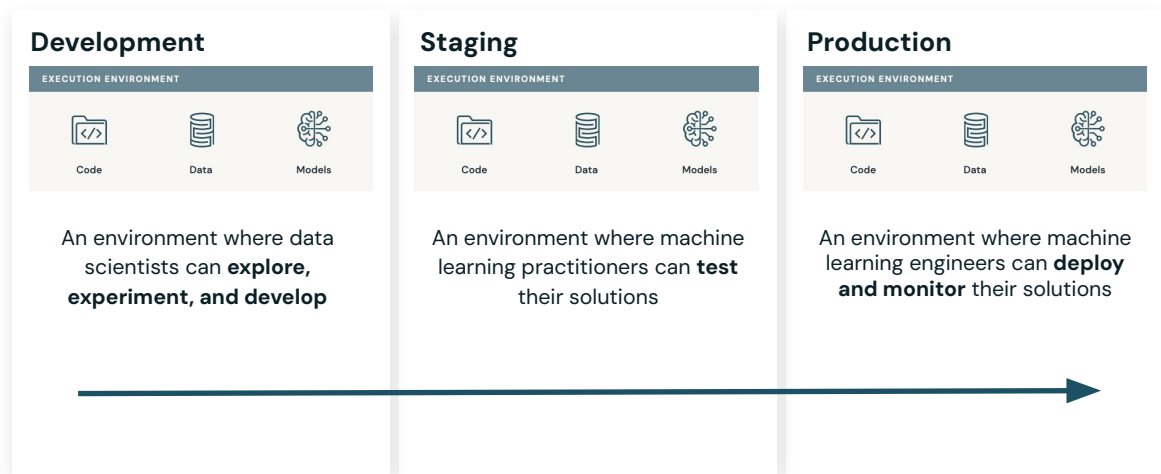


© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

- Databricks advocates for a data-centric approach to machine learning and AI, emphasizing the importance of integrating data pipelines throughout ML projects.
 - Data Intelligence Platform vision = aims for centralized data storage and sharing along with unified security and governance.
- **Strategic Importance of Data-Centric ML:**
 - Requires seamlessly connecting a variety of data pipelines that include data preparation, feature engineering, model training, tuning, and inference.
 - **Benefit = data is easily accessible and manageable** within the ML lifecycle, reducing complexity and enhancing productivity.
 - = data driven decision making
- ***By aligning ML projects with a data-centric approach, organizations can harness the full potential of their data, leading to more effective and scalable ML solutions.***

Multi*-environment Semantics

Defining Development, Staging, and Production environments

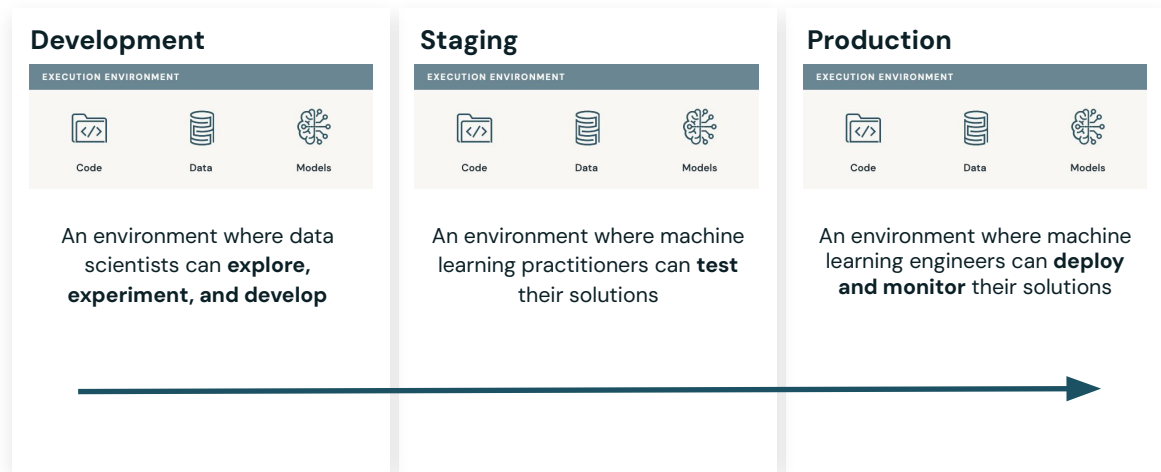


© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

- **Multi-environment Setup = A structured approach** to machine learning projects
 - multiple environments—Development, Staging, and Production—to **optimize and secure the workflow**.
- **Development Environment:**
 - **Purpose:** Where data scientists explore, experiment, and develop their machine learning models.
 - **Activities:**
 - Exploratory Data Analysis (EDA)
 - Writing and refining machine learning workflow code
 - Initial model training and performance evaluation
- **Staging Environment:**
 - **Purpose:** Acts as a testing ground ensure the solution is ready for production.
 - **Activities:**
 - Conducting unit tests, integration tests, and regression tests.
 - **No new development** occurs in this phase; focus is on **validation and testing**.
 - **Transition:** Only after all tests are passed

Multi*-environment Semantics

Defining Development, Staging, and Production environments



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

- **Production Environment:**
 - **Purpose:** Where the solution is deployed for operational use and continuously monitored.
 - **Activities:**
 - Deployment of the ML models into a live setting.
 - Ongoing monitoring and periodic retraining triggered to maintain or enhance model performance.
- **Importance of Environment Differentiation:**
 - Ensures that there is a clear pathway from model conception to deployment.
 - rigorous testing and quality assurance at each stage.
 - Supports a lifecycle approach that minimizes risks and ensures that models are robust and reliable.

Environment Separation

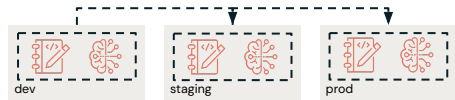
How many Databricks workspaces do we have?

Direct Separation



- Completed separate Databricks workspaces for each environment
- Simpler environments
- Scales well to multiple projects

Indirect Separation



- One Databricks workspace with enforced separation
- Simpler overall infrastructure requiring less permission required
- Complex individual environment
- Doesn't scale well to multiple projects



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

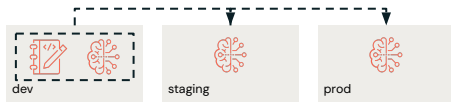
Direct separation manages environments in completely isolated Databricks workspaces, providing simplicity in management, clear boundaries, and scalability for multiple projects, though it requires additional setup and permissions, increasing administrative overhead. Indirect separation uses a single workspace with enforced separation through access controls and naming conventions, reducing infrastructure complexity and resource demands, but it can create challenges in access management and maintaining clear operational boundaries.

Indirect separation is limited in larger teams or multi-project settings, as it can be difficult to maintain clear separation and avoid resource conflicts. Direct separation is recommended for larger teams or organizations managing multiple projects that require strict isolation for security and operational needs, while indirect separation is better suited for smaller teams or projects where ease of management and lower resource usage are prioritized. For this course, indirect separation is used to simplify the learning experience and focus on core MLOps principles without the complexity of managing multiple workspaces.

Deployment Patterns

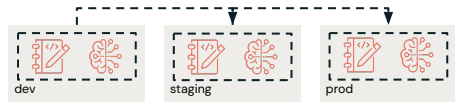
Moving from Deploy Model to Deploy Code

Deploy Model



- Model is trained in development environment
- Model artifact is moved from staging through production
- Separate process needed for other code (inference, monitoring, operational pipelines)

Deploy Code (recommended)



- Code is developed in the development environment
- Code is tested in the staging environment
- Code is deployed in the production environment
- Training pipeline is run in each environment, model is deployed in production



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

There are two main deployment patterns in Databricks: **Deploy Model** and **Deploy Code**, with Deploy Code being the recommended approach. In the Deploy Model pattern, a model artifact created in the development environment is tested in staging before deployment to production, making it suitable when model training is costly or difficult to reproduce. The Deploy Code pattern involves developing training code in the development environment and then moving that same code to staging and production. Models are trained in each environment—development for model building, staging for integration testing on limited data, and production for final training on full datasets. This approach is particularly beneficial when production data access is restricted and supports safer automated retraining, since the code itself is reviewed, tested, and approved. However, it introduces a steep learning curve for data scientists who must hand off code to collaborators. A hybrid approach can also be applied, where code is deployed to staging, the model is trained on full production data, and the trained model is then deployed to production.



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

Thank you for completing this lesson and continuing your journey to develop your skills with us.